

WeebCult: DevOps Architecture and CI/CD Implementation

Mostafa Faik & Derfesh Mariush

October 7, 2026

1 Introduction

WeebCult, an Anime Quiz App built using React, Redux Toolkit, Firebase and an aggregation of external APIs. During the DevOps project, the original app was augmented with a number of other features including CI/CD, automation, IaC, and integration of Automated Security and Code Quality Checks. The goal was not to simply integrate tools in isolation, but rather to design a workflow such that application development, testing, security, IaC, and deployment all work and interact as a unified system.

The setup and final result uses GitHub and GitHub Actions as the development and automation platform, Vitest and Playwright for automated testing, Terraform for infrastructure management, Checkov and Trivy for security assurance, and Firebase Hosting as the deployment target.

2 Architecture and Workflow

This project resides in a GitHub repository, which is the main development platform. On GitHub, the stages of the DevOps workflow run automatically by the GitHub Actions whenever a change is pushed or a pull request is opened. In summary, the general workflow is:

Code change → CI and testing → Security checks → Build → Firebase deployment

The CI workflow involves a couple of validation steps. Dependencies are installed via `npm ci` and Vitest run (unit tests) on relevant areas of application logic. The application is built via `npm run build` and Playwright then runs E2E tests against the application which interact with the app via a browser. Test reports are uploaded as Github Actions artifacts so failures can be reviewed after a workflow has run.

Meanwhile, security tests are performed by Checkov and Trivy: Checkov scans the Terraform code for infrastructure misconfigurations; Trivy scans the repository for potential vulnerabilities. The deployment only occurs if the CI, Checkov and Trivy workflows have all completed successfully on the `main` branch.

3 Continuous Integration and Testing

The combination of CI pipeline, unit testing, building the app, and end-to-end testing. ViTest was chosen as the unit testing framework due to its smooth integration with the JS/Vite ecosystem and rapid feedback. Unit tests are effective for testing parts of application logic, and there's no need to run the whole application.

Yet, the unit tests are not sufficient to verify that all parts of the application are working properly together. So we introduced E2E testing using Playwright. The Playwright tests interact with the application environment just like a user would do, checking navigation, page rendering, interactions with the application, etc

The combination of the two methods gives you complementary coverage. Unit tests are quick and isolated, E2E tests give a top-down approach. The compromise is that E2E tests have to run in a browser, which adds to the CI runtime and introduces additional flakiness for the user interface change.

The production build is also part of the CI pipeline. This is relevant as the test passing does not always imply that one can compile the application. Therefore, the application is built before deploying it so that any build errors can be identified automatically before you get to production.

4 Continuous Deployment

Continuous Deployment is implemented using GitHub Actions and Firebase Hosting. The deployment workflow uses the `workflow_run` event to monitor the completion of the CI, Checkov, and Trivy workflows. Deployment is performed only when the required workflows have succeeded.

After successful validation, the deployment workflow checks out the relevant commit, installs the dependencies, creates a production build, and deploys the resulting application to the Firebase Hosting live channel. This creates an automated path from a successful change on `main` to the production application.

Firebase Hosting was selected because Firebase was already part of the original WeebCult architecture. The application uses Firebase Authentication and Firestore, so continuing to use Firebase for hosting avoids introducing another platform and reduces architectural complexity.

The deployment pipeline itself can be thought of as a kind of continuous deployment as well, as successful commits on the main branch can automatically flow to production without any additional manual step for deployment. The advantage of this is that you can deploy faster and more often. The disadvantage is there is no second-level staging area of deployment and no manual approval.

5 Infrastructure as Code

Terraform is used to manage the infrastructure required by WeebCult. The Terraform configuration defines and configures Google Cloud and Firebase resources, including required APIs, the Firebase project configuration, the Firebase web application, the Firestore database, and Firestore security rules.

Terraform also retrieves the Firebase web application configuration and generates the application's `firebaseConfig.js` file. This connects the infrastructure configuration with the application configuration and reduces the need for manually copying configuration values.

Using Terraform provides a declarative and version-controlled representation of the infrastructure. Infrastructure changes can therefore be reviewed and reproduced using the same Git repository as the application.

This project is making use of an existing Firebase/Google Cloud project rather than provisioning a new Google Cloud project. The choice to follow this route was due to the original application already relying on the existing Firebase project. So the Terraform resources are managing the resources inside that project, and not replacing the whole environment.

This approach has a limitation: the Terraform configuration does not represent every aspect of the complete Google Cloud environment. The existing project itself is treated as an input. Therefore, the infrastructure is reproducible for the resources defined by Terraform, but the configuration does not provide a completely independent recreation of the entire Google Cloud project.

6 Quality and Security Automation

Checkov and Trivy provide automated security assurance during the development process. Checkov evaluates the Terraform configuration and is capable of detecting insecure or misconfigured infrastructure resources, which is important because infrastructure errors can pose security risks even when the application code is functioning correctly.

Trivy performs filesystem vulnerability scanning and generates a vulnerability report. A second Trivy step is configured to fail the workflow when high or critical vulnerabilities are detected, while ignoring vulnerabilities that currently have no available fix. This means that the security tooling is not only informational; security results can also prevent the deployment process from continuing.

These tools complement the functional tests. Vitest and Playwright primarily evaluate whether the application works as intended, whereas Checkov and Trivy address infrastructure configuration and known vulnerabilities.

The automated security scan does not ensure the system is secure. There are a number of known false positives, vulnerabilities for which no fix is available yet, and security issues unique to the specific application that will not be uncovered by a scanner. Human review is still required.

7 Design Decisions and Component Interaction

GitHub Actions was selected as the CI/CD platform because the project is already hosted on GitHub. This provides direct integration with repository events, pull requests, workflow artifacts, and repository secrets without requiring another development platform.

The chosen test tools all have non-overlapping functions, with vitest offering quickly feedback at the unit level and playwright offering a more comprehensive overall end-to-end validation. Similarly, checkov and trivy were chosen for their covering different areas of security concern: configuration of infrastructure and vulnerabilities respectively.

Components communicate using GitHub Actions workflows. Any change in the repo invokes the validation workflows. The CI workflow executes the unit tests, production build and E2E tests, along with Checkov and Trivy for security checks. The Firebase deploys workflow makes sure all workflows execute successfully before deploying the app.

Terraform operates at the infrastructure layer, managing the Firebase and Firestore resources required by the application. Firebase then provides the hosting and backend services used by the deployed WeebCult application.

Thus, the implementation forms an integrated workflow rather than a collection of unrelated tools:

GitHub → GitHub Actions → Tests + Security → Build → Firebase

while Terraform manages the infrastructure supporting the Firebase environment.

8 Limitations and Trade-offs

The main advantage of the solution is automation and repeatability, but these benefits introduce additional complexity. The repository now contains CI/CD workflows, Terraform configuration, security scanning, and multiple testing frameworks that all require maintenance.

The E2E tests provide stronger confidence than unit tests alone, but they are slower and more sensitive to UI changes. Similarly, security scanners provide automated safeguards but cannot replace manual security analysis.

The deployment process is also intentionally simple. It deploys directly into the Firebase live environment once it's validated. No staging environment, no manual approval gate, no blue-green deployment, and no automated rollback plan. These mechanisms could add safety in a larger production system, but they'd also add more infrastructure and operational complexity that's likely to be overkill for a project of WeebCult's scale.

Terraform even comes with a trade off. By making your infrastructure reproducible and version controlled you have to do more, in this case setup and state management compared to configuring Firebase manually. Also, since the Google Cloud project is used as an input, the current IaC configuration won't recreate the project from scratch.

9 Conclusion

The DevOps extension of WeebCult integrates automated testing, continuous deployment, Infrastructure as Code, and security automation around an existing application. GitHub Actions provides the central automation platform, Vitest and Playwright verify application functionality, Checkov and Trivy provide automated security checks, Terraform manages the relevant Firebase infrastructure, and Firebase Hosting provides the production deployment environment.

The biggest advantage of this solution is the collaboration between all these parts. Changes are automatically tested, built and checked before being released, and the relevant infrastructure of the application is expressed as code, and stored in version control with the application. While the solution does not include more complex production procedures like seeding or automated recovery, it presents a workable and reproducible development operations workflow that may be used given the scope and scope of the project.

10 AI Usage

The use of AI has been very minimal for this project, everything from the CI/CD, security/quality control and any other parts of the code which were needed were written by hand, thanks to the efficiency of the tools (Playwright) and extensive documentation (Github actions CI/CD + security/quality control).

The only part which required AI assistance was IaC, due to Terraform having very bad documentation, especially for integrating with firebase. Even then, we only asked for examples and options for Terraform "Resources" (aka API), to understand what was possible to do. If we had used more popular forms of infrastructure such as AWS or Microsoft Azure, we wouldn't have needed to use any AI assistance.

We also used AI to update the documentation, adding more thorough descriptions of setting up prerequisites which we might have forgot about (as we have had them installed for a long time).